

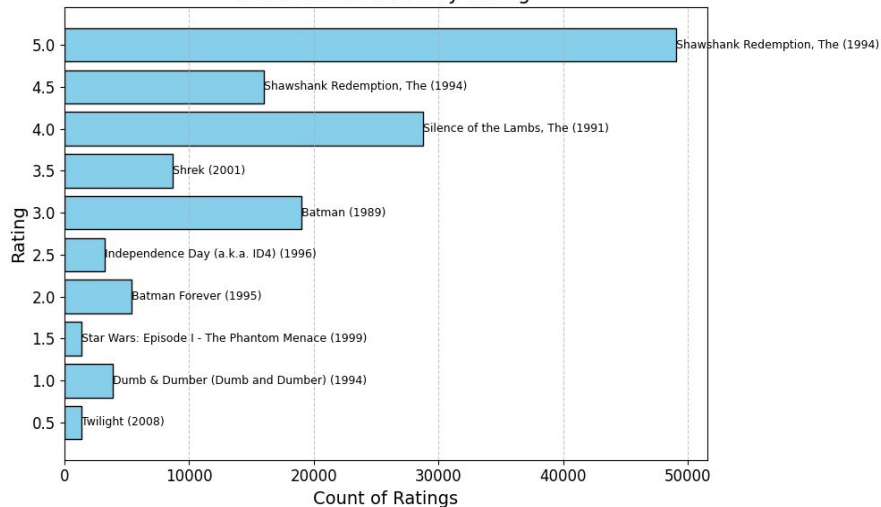
DSCI 4093: Comparison of Recommender Algorithms through MovieLens Data

Charles Hart & Kennedy Johnson

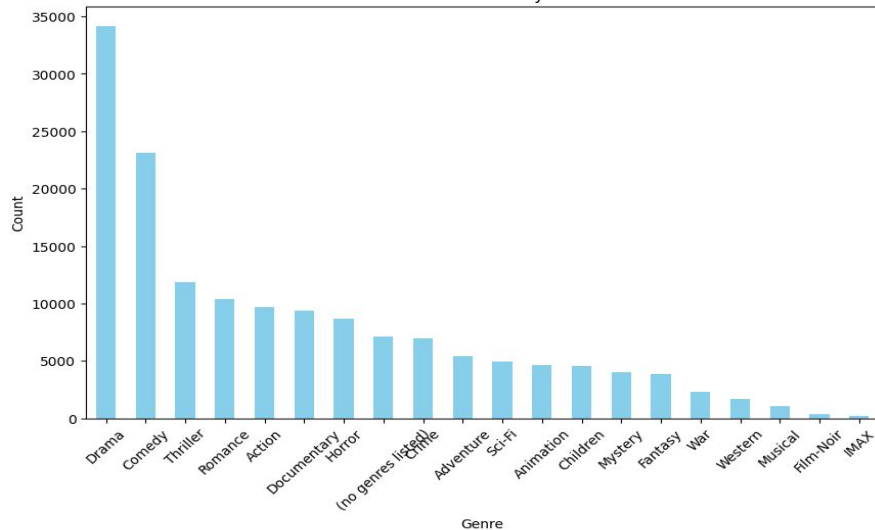
MovieLens 32M Ratings

The dataset contains 32 million ratings from 87,585 movies by 200,948 users gathered by MovieLens (movie recommender site) from 1995 to 2023. Ratings system is out of 5-stars. All of the users have rated at least 20 movies. ~99.8% of dataset is empty aka it's a "sparse matrix". This dataset is from GroupLens, a recommendation systems research lab in CSE at the UMN.

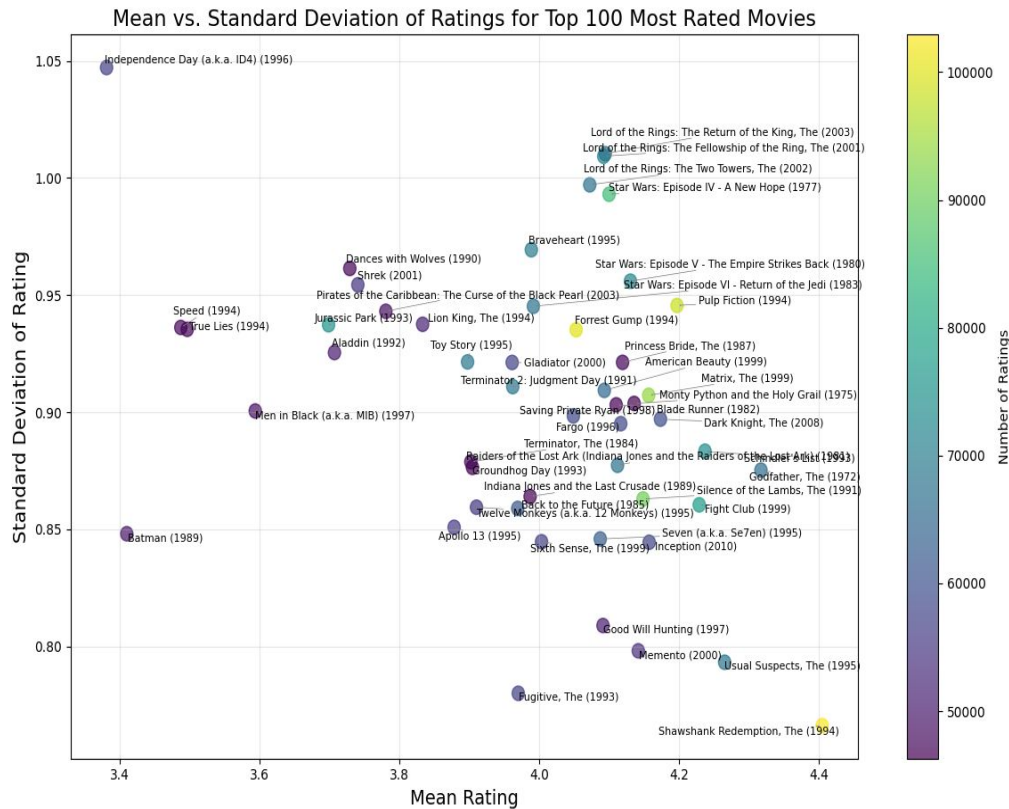
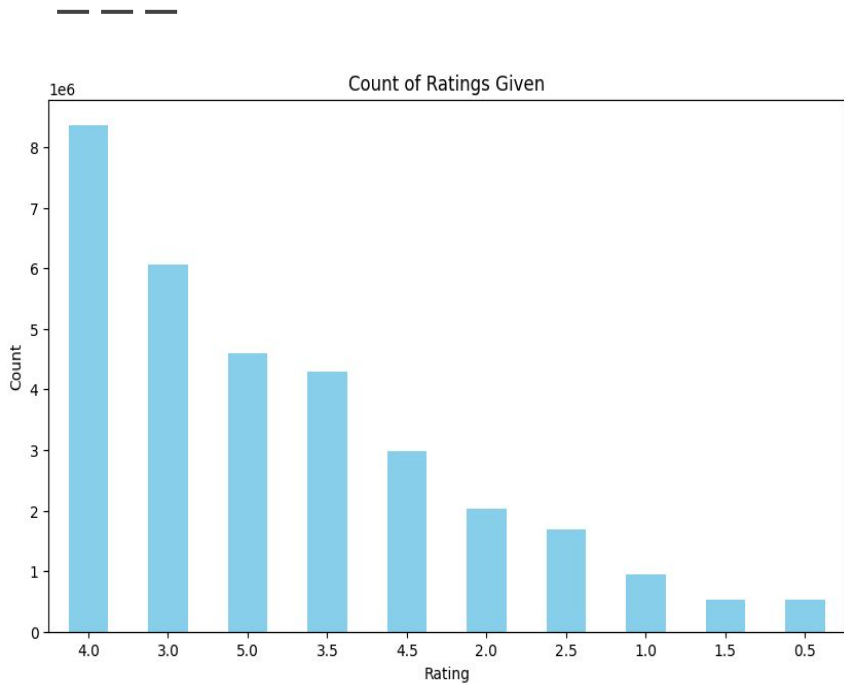
Most Rated Movies by Rating



Number of Movies by Genre



MovieLens 32M Data Analysis



Purpose

- Goal is to see how recommender algorithms compare against one another in different tasks
 - Ranging datasets
 - Large: users with many movies rated (“heavy” users)
 - Medium: users with a moderate amount of movies rated
 - Small: users with not many movies rated (“light” users)
 - Different evaluation metrics
 - RMSE: how Netflix evaluates its algorithms
 - MAE
 - ROC curves
 - Precision/Recall curves
 - AUC values of curves

Algorithms Being Compared

Random (baseline): recommends randomly

Popular: suggests highest rated movies regardless of user

UBCF – User Based Collaborative Filtering: recommends based on user similarity and identifies “neighbors”

IBCF – Item Based Collaborative Filtering: recommends based on item similarity instead of user similarity

SVD – Singular Value Decomposition Column-Mean Imputation: missing values are imputed with column means before SVD

SVDF – Singular Value Decomposition Funk: SVD but avoids imputation and directly factorizes

ALS – Alternating Least Squares: user and item factors are alternated using least squares optimization

PMF – Parallel Matrix Factorization: matrix factorization technique optimized for parallel computation, best used in large systems

The last four algorithms are all matrix factorization methods

R libraries Used for Implementation

— — —

- Recommenderlab
 - “An R Framework for Developing and Testing Recommender Algorithms”
 - Used for implementing algorithms “Random”, “Popular”, “IBCF”, “UBCF”, “ALS”, “SVD”, “SVDF”
- Recosystem
 - R wrapper of the ‘libmf’ library for recommender system using matrix factorization, specifically parallel matrix factorization
 - Only has one specific algorithm but performs well compared to others

Dataset Sizes

Small Dataset: 1000 randomly sampled users with less than 50 ratings per user (~30 ratings per user, 30K rows total)

Medium Dataset: 1000 randomly sampled users with between 50-100 ratings per user (~70 ratings per user, 70K rows total)

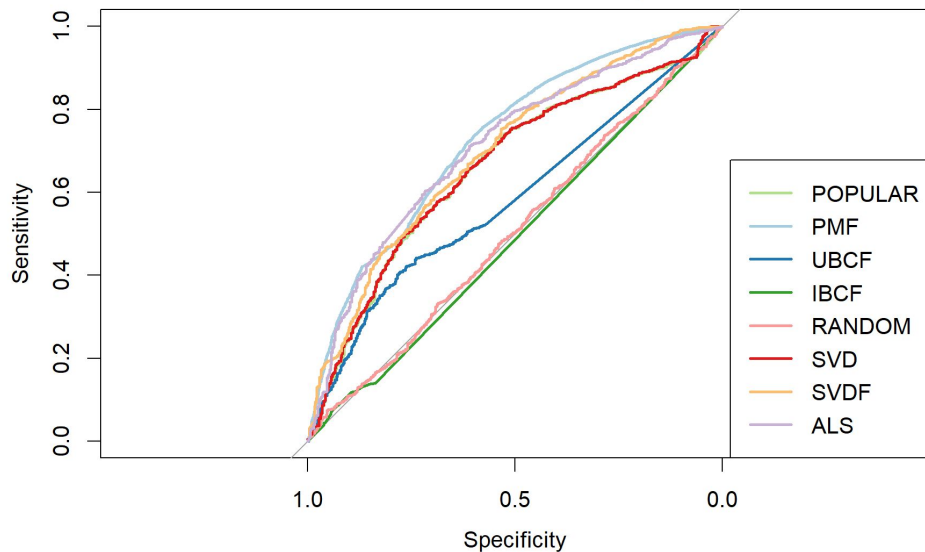
Large Dataset: 1000 randomly sampled users with more than 100 ratings per user (~329 ratings per user, 329K rows total)

The goal of creating these datasets is to identify if certain algorithms or metrics perform better on more sparse datasets, and to see if a more populated dataset can create better recommendations.

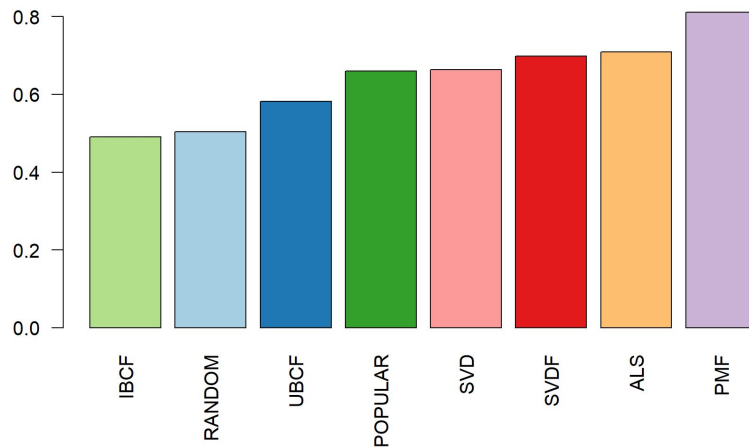
Small Dataset Algos ROC

— — —

ROC Curve for Recommendation Systems - SMALL



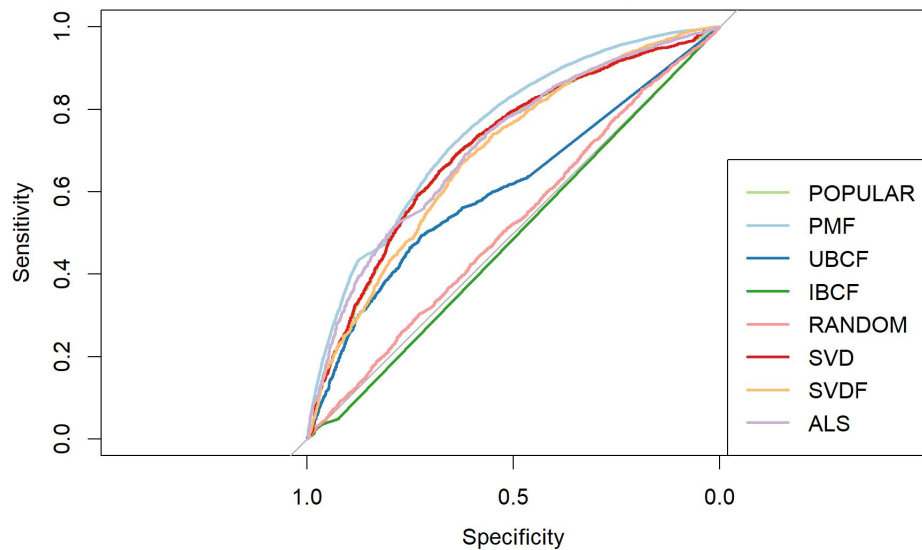
AUC of ROC - SMALL



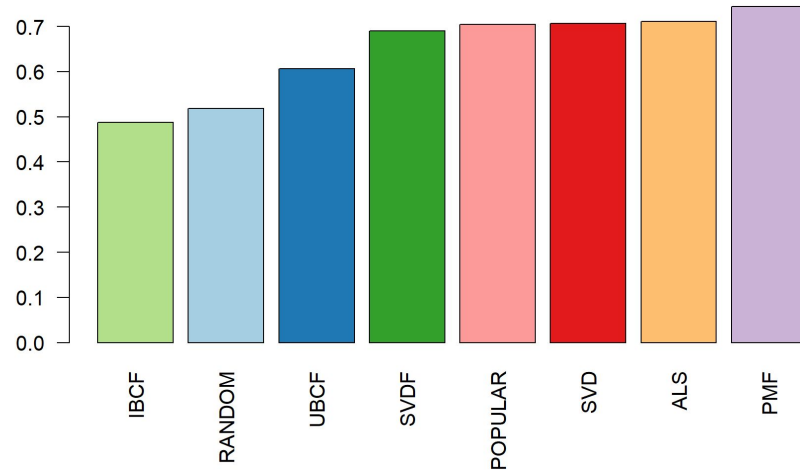
Medium Dataset Algos ROC

— — —

ROC Curve for Recommendation Systems - MEDIUM

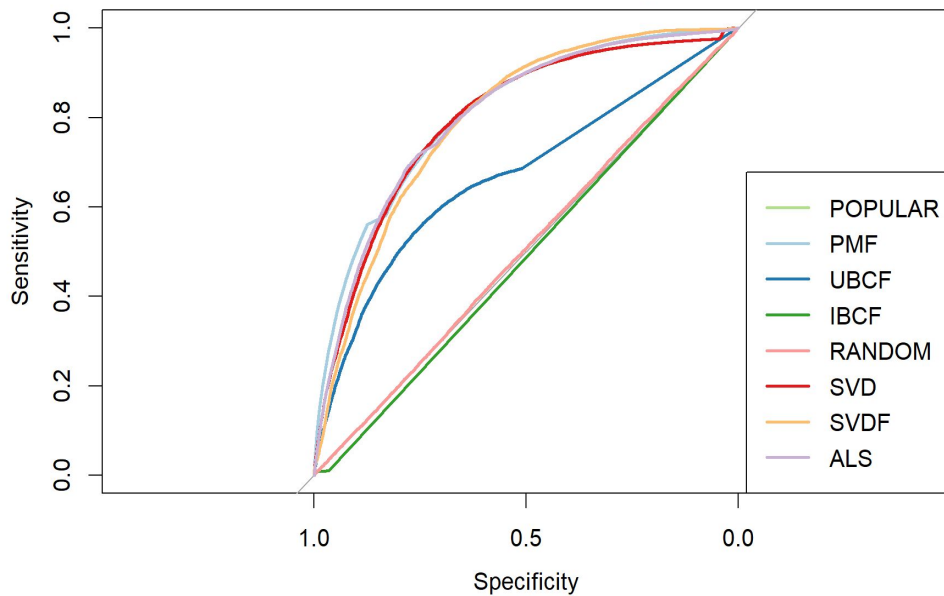


AUC of ROC - MEDIUM

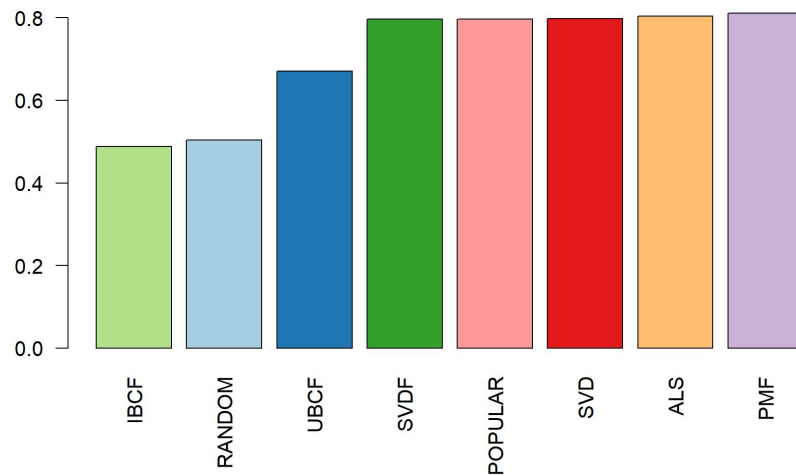


Large Dataset Algos ROC

ROC Curve for Recommendation Systems - LARGE



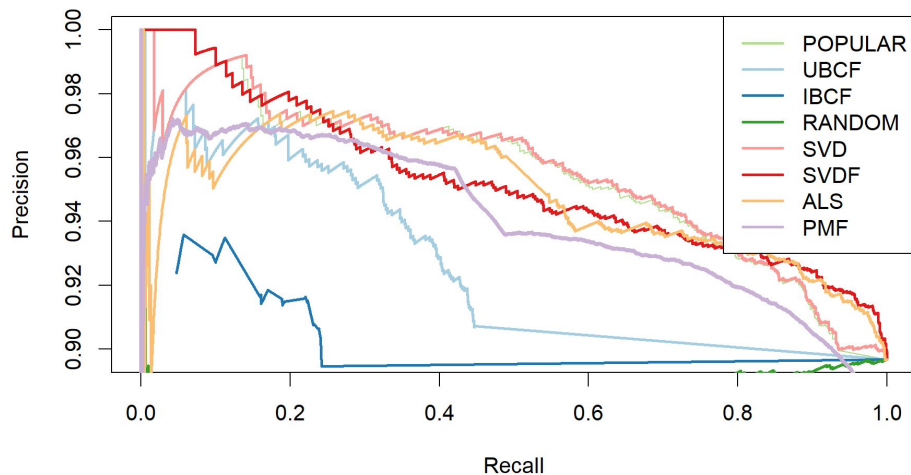
AUC of ROC - LARGE



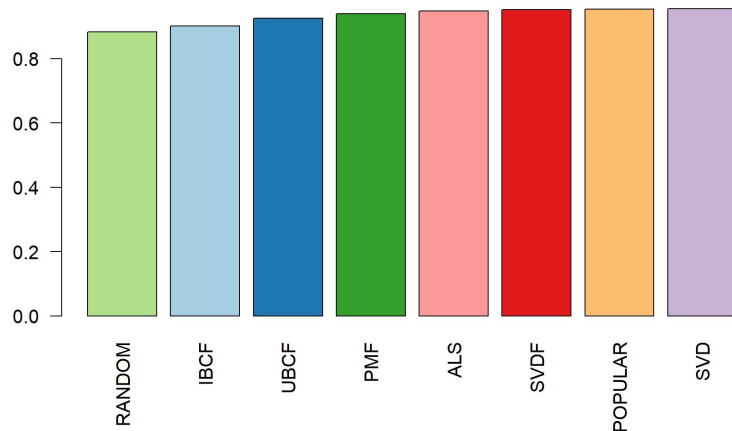
Precision-Recall Curve for SMALL (note y ranges)

— — —

Precision-Recall Curve for SMALL

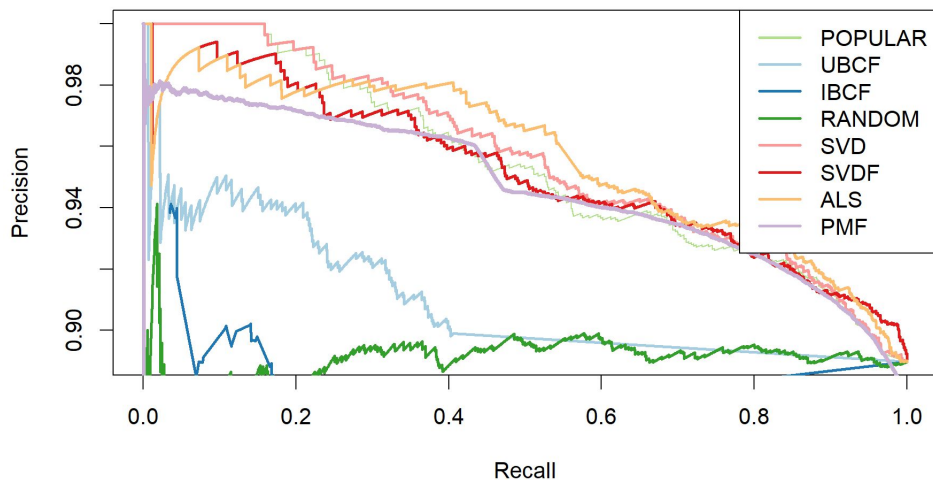


AUC of Precision/Recall - SMALL

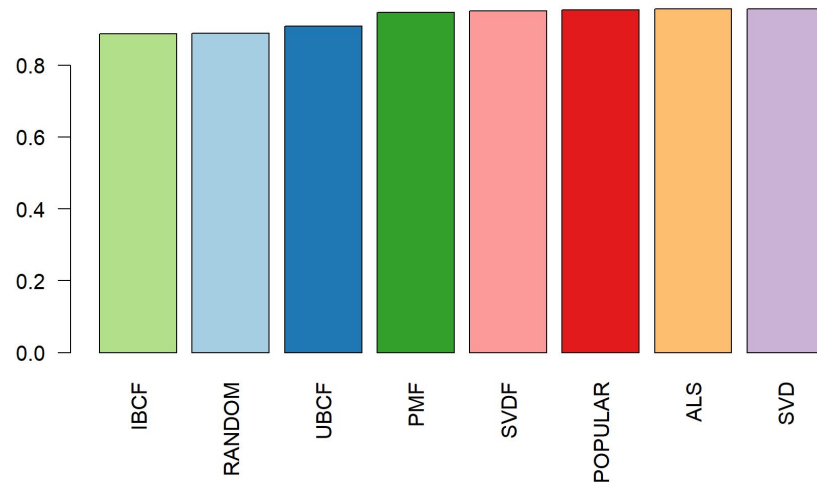


Precision-Recall Curve for Medium

Precision-Recall Curve for MEDIUM

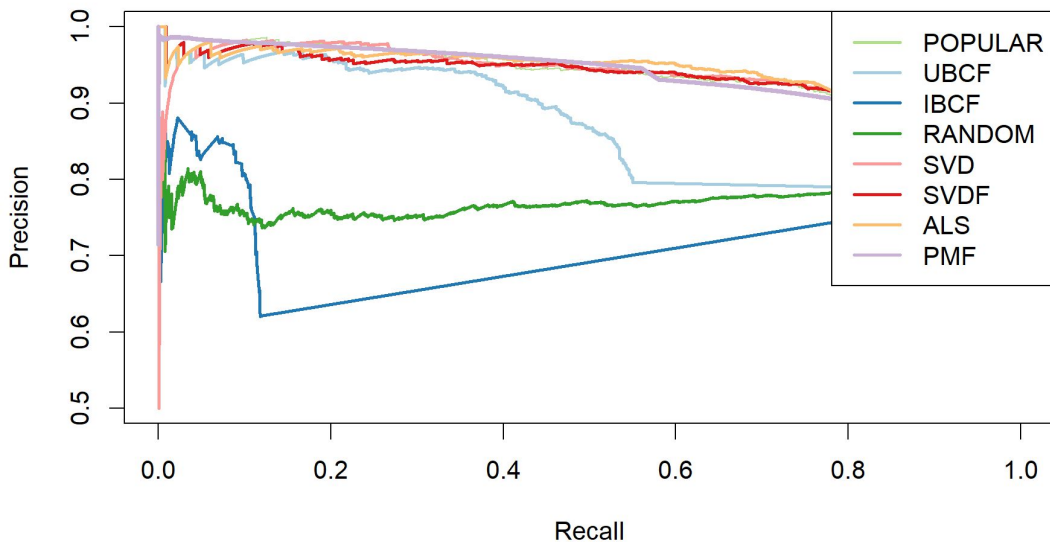


AUC of Precision/Recall - MEDIUM

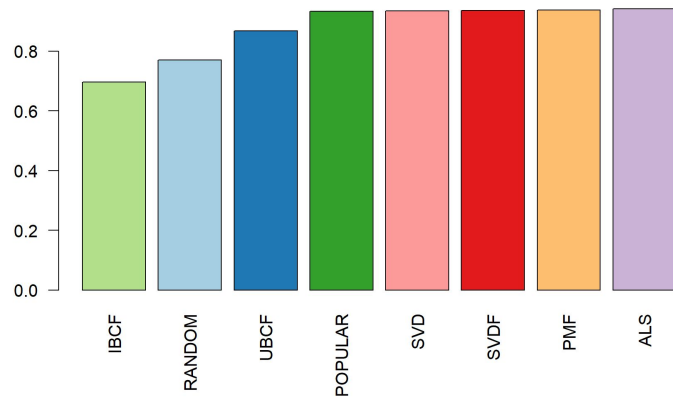


Precision-Recall Curve for Large

Precision-Recall Curve for LARGE



AUC of Precision/Recall - LARGE



Threshold Discussion

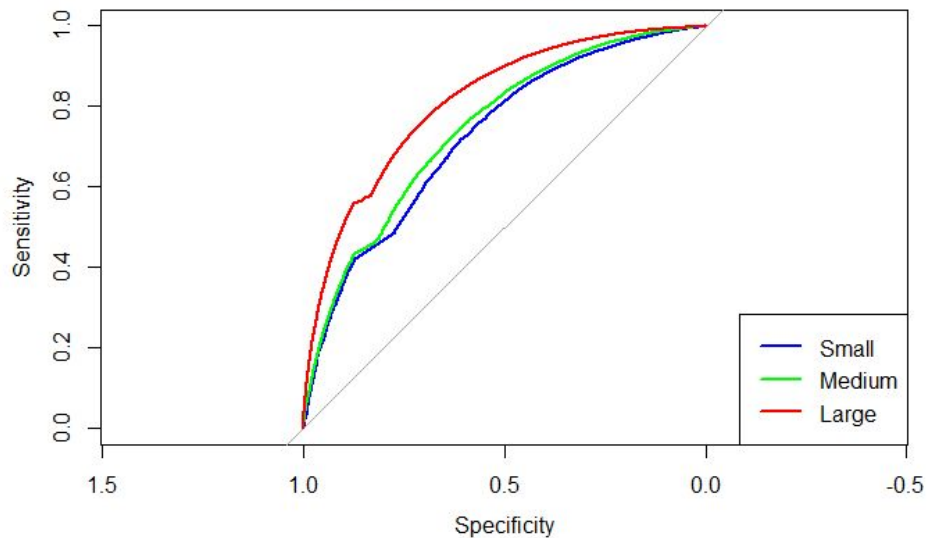
Our response is 1 if ≥ 3 rating, 0 if < 3 rating, so that is why the precision/recall curves are so good, most of the time the algorithm is in a “ballpark” of the correct rating.



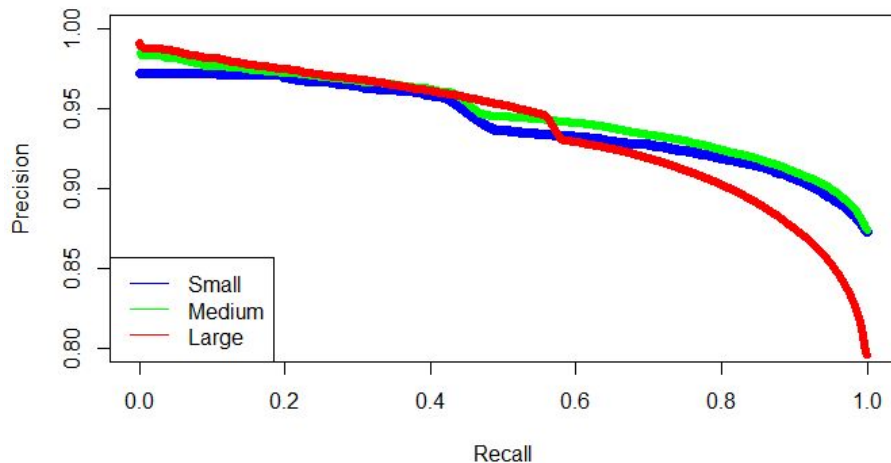
Parallel Matrix Factorization Curves by Dataset Size

-- --

Comparison of ROC Curves



Comparison of PR Curves

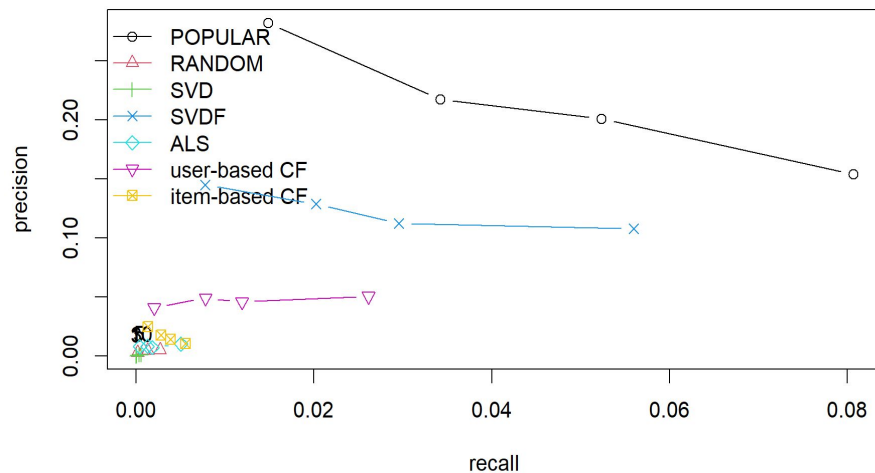
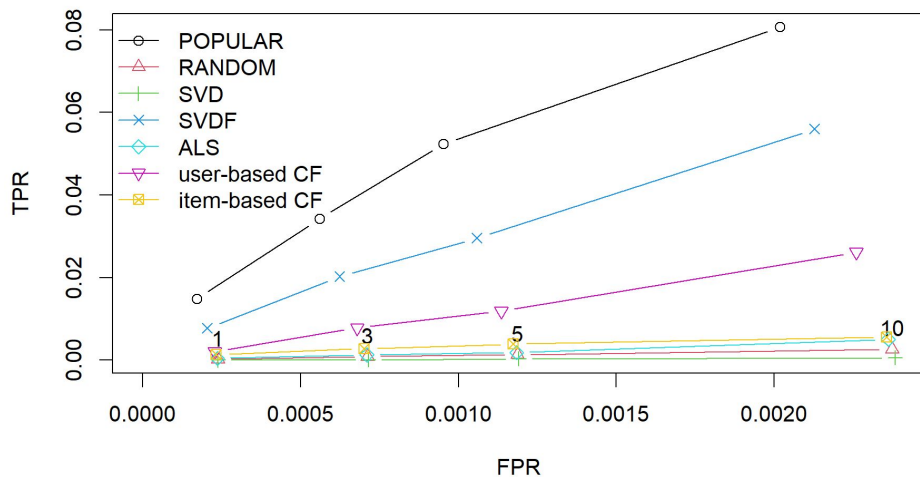


How does the performance of algorithms change as the number of recommendations increases?

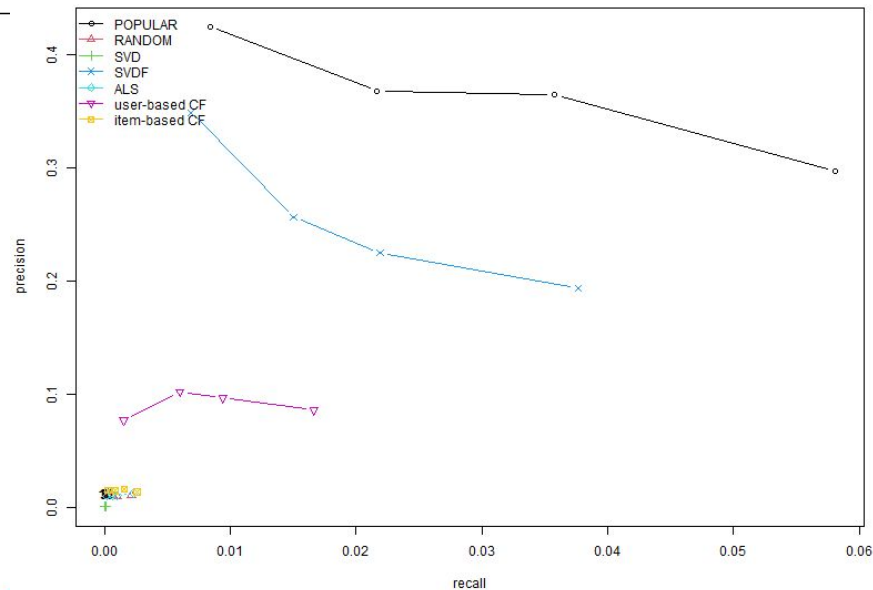
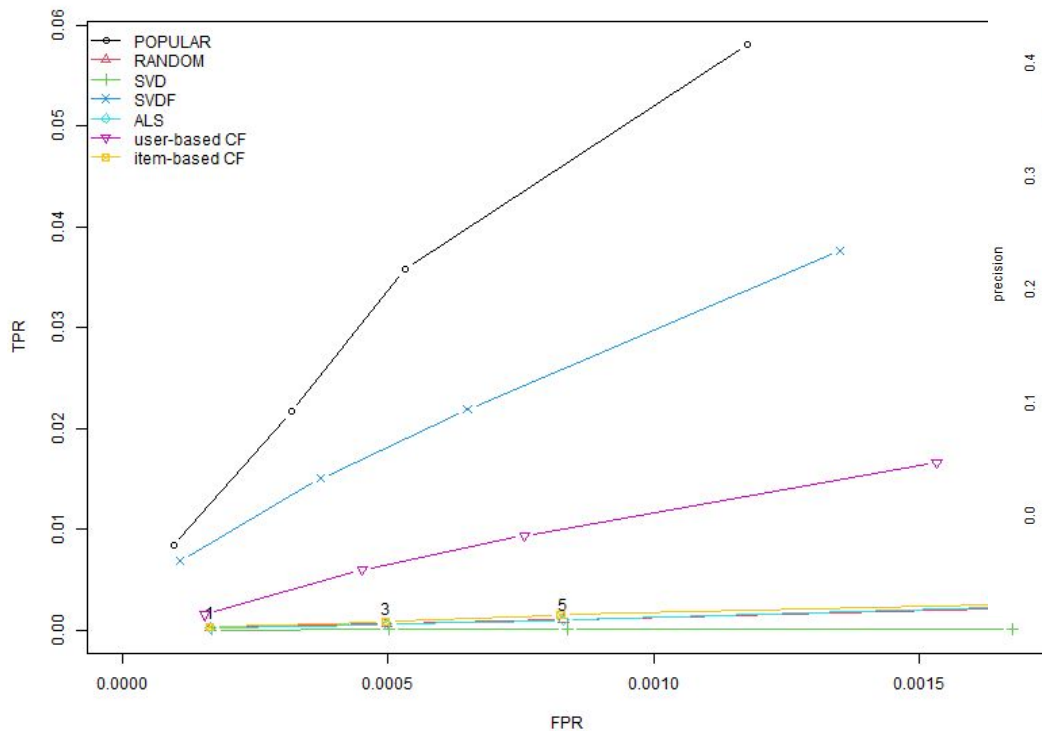
Recommenderlab has a “top N” recommendation list, where the selected algorithm will recommend multiple movies to a user. I selected my “top N” list as (1, 3, 5, 10), where each algorithm will recommend (1, 3, 5, 10) movies to a user, which I then can evaluate how well their recommendations performed. (>3 is a good rating)

Small Ratings TPR/FPR and Precision/Recall Comparison

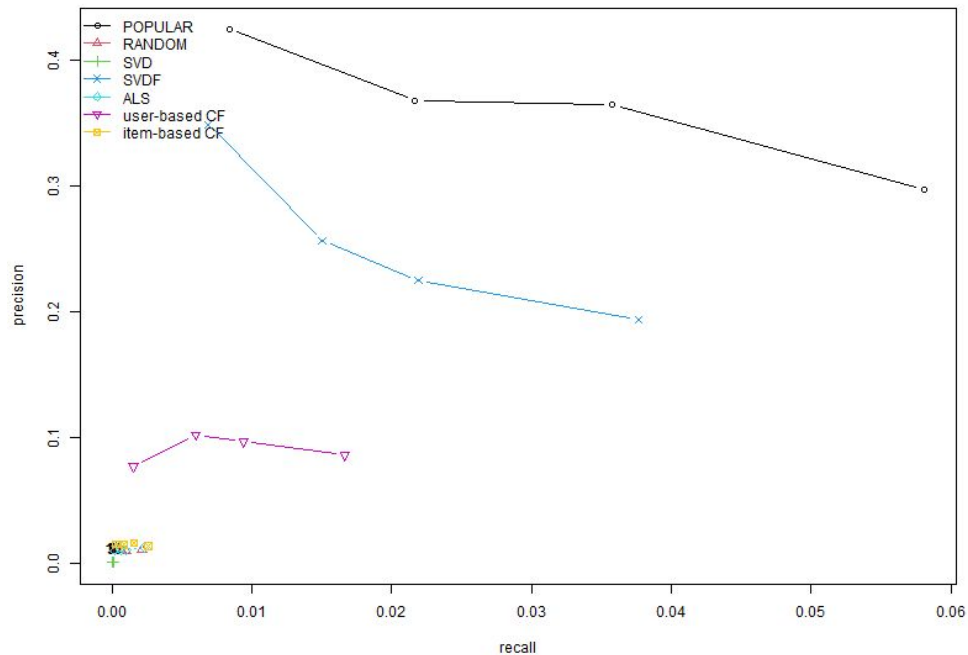
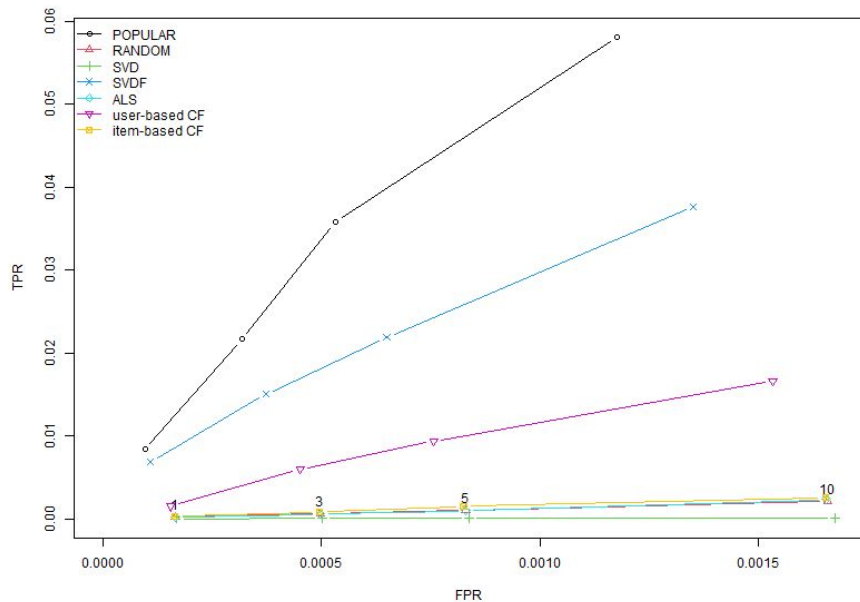
-- -- --



Medium Ratings TPR/FPR and Precision/Recall Comparison



Large Ratings TPR/FPR and Precision/Recall Comparison



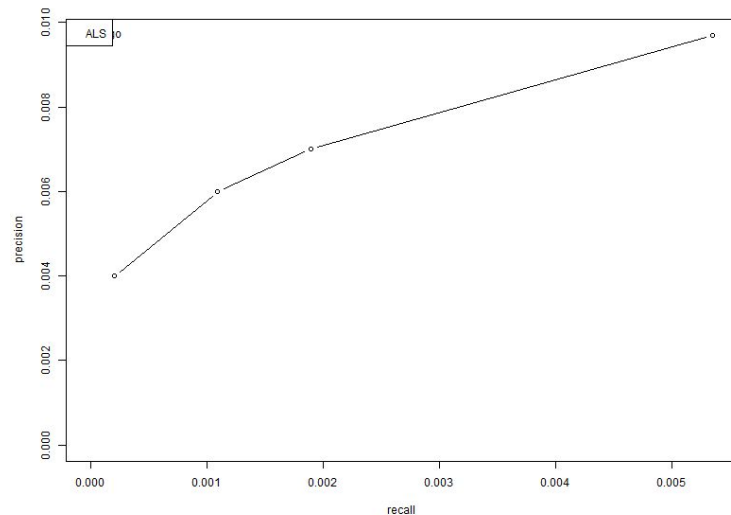
TPR/FPR Discussion

— — —

Systematic increase in TPR and slight increase in FPR as the number of recommendations increases.

Giving more recommendations to the user is desirable, recommending a movie that a user doesn't like isn't very costly, where missing a recommendation to a user that they really like is a costly mistake. **Therefore, we want to increase TPR, but don't really care about FPR.**

Note about algos at the bottom (SVD, ALS, IBCF), still increasing, just very slowly, observe the scale of the plot on the right (zoomed in view), same thing for precision/recall curves.



RMSE and MAE Discussion

How was RMSE/MAE determined?

Using 5-fold CV, each algorithm would make a prediction for every movie for every user on the validation fold and if the prediction was > 3 and the actual value was too, then the prediction was correct (and vice versa for < 3)

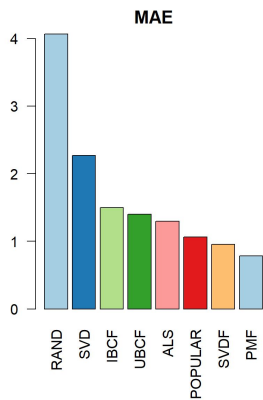
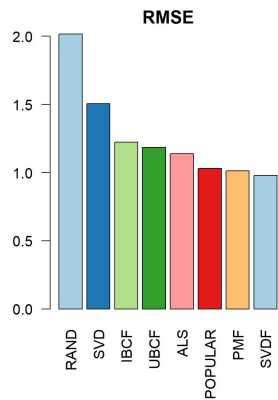
$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

$$MAE = \frac{\sum_{i=1}^n |y_i - x_i|}{n}$$

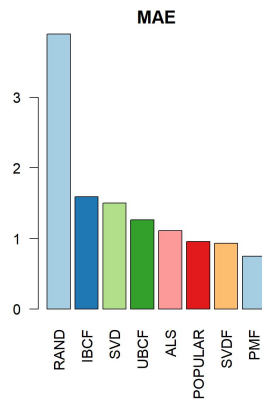
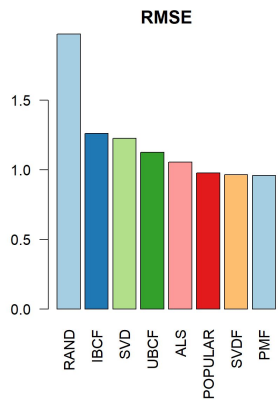
RMSE and MAE Comparison by Algorithm over different sized datasets

-- -- --

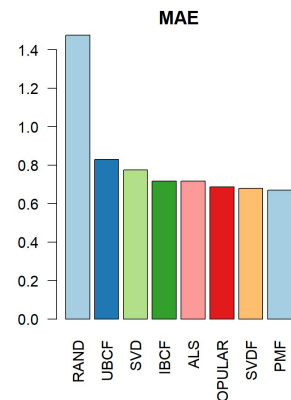
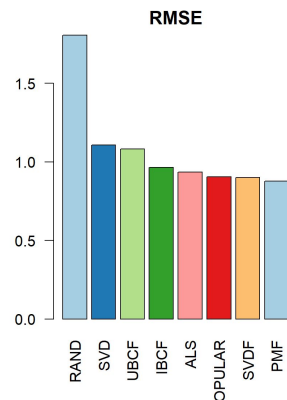
Small dataset



Medium dataset



Large dataset



Discussion of results

— — —

- **Systematic increase in performance as the ratings per user goes up**
 - Large dataset performance > Medium dataset performance > Small dataset performance
 - More training data for algorithm allows for more accurate recommendations made to users -> algorithms with less sparse datasets give better results
- **SVDF and PMF were the only algorithms to outperform the “POPULAR” algorithm in terms of RMSE/MAE**
 - Both algorithms performed well on all sizes of datasets
- **IBCF went from poor performance in small ratings dataset (<100 reviews per user) to better performance as the # of reviews per user increased, (more items, more training data)**
 - If there is a populated dataset, IBCF might be preferred over another algorithm
 - If there is many users, then UBCF would perform better, but we didn't have the computation power to use many users

Conclusion

- Since all algorithms improved as the density of ratings increased, it is desirable for movie sites to push users to rate movies
 - Netflix prompts users at the end of movies for a “thumbs-up” or “thumbs-down”
- Since predictive power increased as the number of recommendations were given, it is beneficial for movie sites to push as many recommendations as possible
 - Movie sites have long lists of recommendation to the user
- Which algorithms worked best?
 - SVDF and PMF both performed well in terms of RMSE and MAE and all of the various sized datasets
 - Matrix factorization methods (SVDF, PMF, ALS) all performed well-work well on all datasets, IBCF/UBCF did not

Citations

MovieLens Dataset

F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1–19:19. <https://doi.org/10.1145/2827872>

Recommenderlab

Hahsler M (2023). *recommenderlab: Lab for Developing and Testing Recommender Algorithms*. R package version 1.0.6, <https://CRAN.R-project.org/package=recommenderlab>.

Hahsler M (2022). “recommenderlab: An R Framework for Developing and Testing Recommendation Algorithms.” arXiv:2205.12371 [cs.LG]. [doi:10.48550/ARXIV.2205.12371](https://doi.org/10.48550/ARXIV.2205.12371).

Recosystem

Qiu, Y (2023). *recosystem: Recommender System using Matrix Factorization*. R package version 0.5.1, <https://cran.r-project.org/web/packages/recosystem/index.html>

This project is not associated with GroupLens

Appendix

Disclaimers

- Sampled only 3,000 users total across all data
 - Size significantly lower but had to truncate due to processing time
 - Some methods like IBCF, UBCF perform better trained on larger datasets
- Metrics across libraries and hyperparameters can cause discrepancies
 - Algorithms were checked across libraries with negligible differences on RMSE, MAE due to hyperparameter differences

Recosystem Hyperparameters

— — —

- Dimensionality of Latent Factors: 10
- Learning Rate: 0.1
- Regularization parameter for user factors: 0.1
- Regularization parameter for item factors: 0.1
- Number of iterations: 20

Recommenderlab hyperparameters (defaults resulted in better performance)

— — —

For SVD, SVDF, ALS

- `k` (number of latent factors): 20 (by default).
- `lambda` (regularization parameter): 0.1.
- `iterations` (number of iterations): 10.

UBCF (User-Based Collaborative Filtering)

- `nn` (number of nearest neighbors): 30 (default value, meaning it will use the 30 nearest neighbors).
- `method` (distance measure): "Cosine" (default value, which computes similarity between users using cosine similarity).
- `normalize`: TRUE (default value, indicating that the ratings are normalized before computing similarity).

IBCF (Item-Based Collaborative Filtering)

- `nn` (number of nearest neighbors): 30 (default value, meaning it will use the 30 nearest neighbors).
- `method` (distance measure): "Cosine" (default value, which computes similarity between items using cosine similarity).
- `normalize`: TRUE (default value, indicating that the ratings are normalized before computing similarity).

No hyperparameters for Popular, Random algorithms